

FRED TALKS

10th July 2026

CONTENTS

Blog about things you don't understand yet

SEANGOEDECKE.COM RSS FEED · 1573 WORDS

What I Learned About Billionaires at Jeff Bezos's Private Retreat

NOAH HAWLEY · 2436 WORDS

Search has its own bitter lesson

DOUG TURNBULL · 1012 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

[Daily article] July 10: Gu Yanwu

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 406 WORDS

Blog about things you don't understand yet

SEANGOEDECKE.COM RSS FEED · 07 JUL 2026 · [SOURCE](#)

Every post I publish represents at least two things I've learned: the thing that prompted me to write the post, and the thing I learned in the course of writing it. If I don't learn anything new while I'm writing, it's not interesting enough to publish.

Typically I learn way more than two things. For instance, in my [o3 geoguessr](#) post, I started out with the idea that most AI prompts probably don't work, and I ended up learning that newer OpenAI models have lost o3's ability to geolocate. That's interesting! In my most recent post on [C2PA](#), I started out with the idea that C2PA requires near-universal adoption, but I learned a *ton* of things about PKI, managing private keys on local devices, how C2PA actually works, and so on. In my post on the [Luddites](#), I started out with the idea that the Luddite movement was fundamentally decentralized, but ended up fascinated by Luddite culture (which was far more elitist, misogynist, and violent than the pop-Luddism books describe). I could do this for every single post on the blog.

Taking a position

I think the core reason this works is that **every single one of my blog posts argues a point**. I never publish a post that just gives some scattered thoughts on a topic, or a post that only says "yes, I agree with this other article". If I write a draft that nobody sensible could disagree with, I scrap the draft. Making sure that everything I write is at least minimally controversial is a forcing function: it forces me to think about what the most interesting part of my position is, and it forces me to do enough research to defend it against the obvious criticisms.

This is contrary to a lot of advice I read about blogging, which encourages the aspiring blogger to treat their posts as a form of unstructured self-expression. If unstructured self-expression is what you want to do, that's cool. The point of having a blog is that you get to write what *you* want. However, this advice isn't as helpful as it sounds.

Before I was in tech, I was a philosophy grad student. But before *that*, I was a poet. One thing you learn when you try to write poetry is that it is way easier to write to a restrictive structure than it is to simply "write what you feel". This should be obvious when you actually think about it. The task of a poet is to repeatedly choose the next word. Writing to a structure (typically rhyme or meter) narrows that choice to a small set of words, instead of the entire English language. It's the same with blogging. Forcing yourself to write about specific, potentially-controversial points makes consistently writing easier, not harder.

Writing, thinking, and research

Writing is the best way to think clearly about a topic. It's easy to believe you understand something when you're just turning it over in your head. When you have to condense that down into words, you find out exactly how much you do or don't understand. I am constantly having moments where I type something, stop myself, and think "wait, that can't actually be right", or "is that really true?"

By the time I write my way to the end of the post, I'm usually thinking so much more clearly about the topic that my conclusion paragraph is way better than my introduction. In fact, I've picked up the habit of going back and immediately rewriting the first paragraph as part of my first-draft process, because I know I'm going to end up doing it anyway.

I also change my mind a lot while I write. [Here are a bunch of examples of posts](#) where I began writing them with the opposite opinion to the one that eventually made it into the post. I think this is a good sign, and I hope I never stop doing it. You should be researching and thinking about every post you write, and that means you should frequently learn new things that change your mind.

Because of all this, I deliberately choose to write blog posts about things I don't yet quite understand but would like to, like [LLM steering](#), Stripe's [Tempo](#) blockchain, [C2PA](#) and [watermarking](#), space [cooling](#), [interaction models](#), LLM inference [internals](#), and so on. This is great for me, because I learn a lot. Is it great for my readers?

Is blogging to learn irresponsible?

I sometimes worry that I should only be writing about areas I already know very well, like [tech company dynamics](#) or [working in large codebases](#), rather than presenting myself as an authority on fields I'm actually still learning. Should I let historians of the Luddites write about Luddism, Web3 engineers write about blockchains, and so on? I think this is acceptable for three reasons.

First, it's sometimes easier for a beginner to write an introduction to a field than for an expert. Experts routinely [overestimate](#) the knowledge of the general public, and have often internalized the reasons why their field is important so deeply that they struggle to express them. I think my [explainer posts](#) are valuable because I always spend the first chunk of the post talking about *what the original problem is* before I get into the technical solution.

Second, sometimes the public consensus on a topic is just plain wrong, to the point where even a little bit of research is enough to demonstrate why. Many of my posts I'm proudest of have been along these lines: arguing that the "500ml per prompt" water usage figure for LLMs was

ludicrous, or that the popular Apple “Illusion of Thinking” paper was tracking persistence, not reasoning, that GPUs live longer than three years and the AI companies have large profit margins on inference, and so on.

Third, I try to make it clear on my blog who I am and what my credentials actually are. Even if it’s not explicitly described in the post, I have my real name and resume available on my /about page, so I don’t think a careful reader could be easily fooled into thinking I’m an expert on 19th-century England or space physics or LLM economics or anything like that.

Feedback

Even if nobody reads what you write, writing is still a good discipline for getting your thoughts in order. But another big reason why writing is a great learning tool is that **you can get feedback**.

I think it’s obvious why this is useful, but I do want to make two points about feedback. First, if you do make your posts public, you need to have a pretty thick skin. People on the internet often fall over themselves to come up with the most cutting criticism or the harshest dunk. This goes double if you take my previous advice and try to write posts that make a clear, controversial point about a subject you’re learning. If you’re the kind of person whose whole day is ruined when a stranger is cruel to them, you might want to keep your blogging private or only share it among friends.

Second, even if your blogging is private, **you can get feedback from LLMs**. Like humans, LLMs will often give junk feedback. In my experience, OpenAI models will always tell me to moderate my claims or add caveats and hedges until I’m not saying anything at all. Sometimes their criticism will be straight-up wrong. But — particularly about technical topics — LLMs are great at pointing out areas you’ve genuinely misunderstood, and they’re far kinder than the average Lobsters or Hacker News commenter.

Conclusion

I’m pleased and grateful that people enjoy reading my posts, but even when nobody did, I still got a lot of value out of blogging. I write as a method of thinking more clearly, as an excuse to do research on topics I want to learn about, and as a way of getting feedback.

If you'd like to try it yourself, I suggest watching for these two things. First, you should be changing your mind a lot as you write. If not, you probably aren't doing enough research. Second, your first draft's conclusion should be much tighter and more expressive than its introduction. If not, you probably haven't learned anything from the writing process, which means the draft can be scrapped.

I strongly recommend this practice to anyone with an interest in writing. You will see the benefits even if you don't publish any of your writing on the internet, particularly now that you can get good technical feedback by pasting your post into a LLM¹.

1. For what it's worth, I've fiddled with careful "review prompts" and it's basically as good to just write "review, please:" and paste your article.

↩

What I Learned About Billionaires at Jeff Bezos's Private Retreat

NOAH HAWLEY · 02 MAY 2026 · [SOURCE](#)

For the richest men on Earth, everything is free and nothing matters.



(Illustration by Tim Enthoven)

At the end of Paul Thomas Anderson's 2007 movie, *There Will Be Blood*, Daniel Day-Lewis's oil-baron character, old now and richer than Croesus, beats Paul Dano's preacher to death with a bowling pin. Dano's Eli Sunday, a nemesis of Day-Lewis's Daniel Plainview during his seminal, wealth-building years, has come to sell Plainview the oil-rich land that he once coveted. But

Plainview doesn't need the land anymore, because—as he explains in one of the most famous monologues in modern cinema—he has sucked out all the oil hidden beneath it from an adjoining property, like a milkshake.

Desperate for money, Eli begs for a loan. Instead, Plainview chases him around a bowling alley and murders him with great enthusiasm. Once it's over, a butler comes to see what all the noise was about. "I'm finished," Plainview yells.

No matter how many times I watch that movie, and I watch it a lot, I have never once taken those words to mean *I'm done for. There will now be consequences for my actions*. Quite the opposite: They mean that Plainview has completed his journey, through the acquisition of wealth and power, to a realm outside the moral universe. He's finished, in other words, pretending that the rules of human society apply to him.

In 2018, I was a guest at Jeff Bezos's Campfire retreat in Santa Barbara, California. It's an annual event in which the Amazon founder invites 80-plus guests—celebrities, artists, intellectuals, and anyone else he thinks is interesting—to spend three nights at a private resort. I had recently been approached by Amazon about moving my film-and-television business over from Disney, and although I had declined (or maybe *because* I had declined), Bezos's team invited me to Campfire, perhaps keen to impress me with the power of his reach.

From the March 2024 issue: The rise of techno-authoritarianism

On a warm October Thursday, a fleet of private jets was dispatched to airports in Van Nuys and New York to shepherd guests to Santa Barbara in style. At that point I had only a vague sense of who else was coming—famous people, rich people, influential people, and me. A guest list, I was told, would be given to us once we arrived. Families were invited; an on-site nanny would be provided *for each child*.

So my wife and I got our two children from Austin to Los Angeles and took a 45-minute jet ride north, with a television mogul and a comedian on board. Bezos had bought out the entire Biltmore resort for the weekend, as well as the beach club across the street. He had brought in a security firm from Las Vegas to ensure our safety and privacy. Even the weather felt expensive, and when we were shown to our rooms, the designer gift bags we found were filled with luxury goods.

Alexandra Petri: The 10 things the Bezoses are almost certainly grateful for each morning

Each morning, we gathered in a lecture hall to hear presentations. If you've ever seen a TED Talk, you understand the format. The year I went, a sitting Supreme Court justice was interviewed, and a neurologist talked about technological advances in prosthetics. In the

afternoons and evenings, we were encouraged to exchange ideas over drinks and four-course meals, with no set purpose—to network, in other words, with some of the most rarefied talent on Earth. The most common question I heard was “Why am I here?”

“Why am I here?” asked the 1980s hair-metal singer. “Why am I here?” asked the Pulitzer Prize–winning novelist, the famous anthropologist, the presidential historian. Only the movie stars and the billionaires didn’t ask: They had done this kind of thing before. It turns out there is a circuit of idea festivals. Many tech billionaires host one, and if you find yourself on the right list, you can spend much of the year traveling the world, eating Wagyu, and discussing how to make the world a better place with the most famous talk-show host in history.

That’s how the weekend started. Here’s how it ended: My wife broke her wrist slipping on wet grass, and both children and I came down with hand, foot, and mouth disease. This is not a joke. One of us went home with her arm in a sling; the other three developed itchy, painful red blisters all over our faces and extremities. If you’re looking for a sign from God as to whether hanging out with the richest man on Earth is right for you, pay attention when he sends you not one plague, but two. Suffice it to say we have never been back to Campfire, nor have we ever been invited.

At drinks on the second night, the head of a major talent agency asked me what I thought of the weekend. I said, “I’ve spent my whole career trying to figure out how the world works. I didn’t realize I could just come here and ask the people who ran it.” On some level I was kidding. The lead singer of an alt-country band didn’t run the world, nor did a noted author who would later be accused of impropriety. But finding myself at that resort by exclusive invitation, I now knew exactly what people meant when they talked about the elite.

Sitting in the lecture hall, pencils out, listening to a famous chef explain his humanitarian work, it was easy to feel like the solution to the world’s problems lay within our grasp. And yet, looking around at faces I had only ever seen in a magazine or on-screen, I had an unsettling revelation: This is the hubris of accomplishment. To be declared a genius at one thing is to begin to believe you are a genius at everything.

Here we were, 80 individuals with a combined net worth that was greater than a small city’s yet infinitesimal compared with the wealth and dominion of our host. How did he view this exercise—as a first step toward changing the world, or as a performative display of his reach and influence?

Bezos was everywhere that weekend—in a tight T-shirt, laughing too loudly, arms thrown around his teenage sons. He had recently become the world’s second centibillionaire, his net worth hovering somewhere around \$112 billion, about half of what it is today. That number,

previously unimaginable, had made him unique on a planet of 8 billion people, and you could feel it in the room. Even the richest and most famous among us were drawn to the energy of this impossible wealth.

Martin Baron: Where Jeff Bezos went wrong with *The Washington Post*

Though we didn't know it at the time, Bezos's first marriage would be over a few weeks later. My defining impression of his wife that weekend was sadness, even though Bezos made a big show of performing the role of family man. In hindsight, it is that performance that sticks with me. The Jeff Bezos of 2018 acted as if he still believed that people's impression of him mattered, that his financial and social value could be affected by negative publicity. He still believed that his actions had consequences. He had not yet freed himself—the way Daniel Plainview freed himself—from the rules of men.

Eight years later, Bezos and two of the world's other richest men—Mark Zuckerberg and Elon Musk—have clearly left the world of consequences behind. They float in a sensory-deprivation tank the size of the planet, in which their actions are only ever judged by themselves.

The closer I've gotten to the world of wealth, the more I understand that being truly rich doesn't mean amassing enough money to afford superyachts, private jets, or a million acres of land. It means that everything becomes effectively free. Any asset can be acquired but nothing can ever be lost, because for soon-to-be trillionaires, no level of loss could significantly change their global standing or personal power. For them, the word *failure* has ceased to mean anything.

Adam Serwer: How America chose not to hold the powerful to account

This sense of invulnerability has deep psychological ramifications. If everything is free and nothing matters, then the world and other people exist only to be acted upon, if they are acknowledged at all. This is different from classic narcissism, in which a grandiose but fragile self-image can mask deep insecurity. What I'm talking about is a self-definition in which the individual grows to the size of the universe, and the universe vanishes. Asked recently if there is any check on his power, President Trump—himself a billionaire, and by far the richest president in American history—said, “Yeah, there is one thing. My own morality. My own mind. It's the only thing that can stop me.” Not domestic or international law, not the will of the voters, not God or the centuries-old morality of civic and religious life.

Decades of research in developmental psychology have shown that moral reasoning develops through consequences—not punishment, necessarily, but experiencing the effects of your actions on others, receiving honest feedback, having to accommodate reality as it actually is rather than as you wish it to be. It's not that the wealthy become evil; it's that their environment stops

teaching them the things that nonwealthy people are forced to learn simply by living in a world that pushes back. When you can buy your way out of any mistake, when you can fire anyone who disagrees with you, when your social circle consists entirely of people who need something from you, the basic mechanism by which humans learn that other people are real goes dark.

Thomas Chatterton Williams: The very powerful men who think introspection is dumb

When Peter Thiel said, “I no longer believe that freedom and democracy are compatible,” he wasn’t talking about your freedom. He was talking about his own. You don’t exist. When Musk took a chainsaw to the federal government as part of the inside joke he called DOGE, he did so with the air of a man who believed that nothing matters—poverty, chaos, human suffering. He was having fun. It didn’t even matter that the entire destructive exercise ultimately yielded no practical financial gains. For him, the outcome was a foregone conclusion: He could only win, because losing had lost its meaning.

Since the 2024 election, there has been a philosophical shift on the right, and especially among tech billionaires, to vilify the idea of empathy. Musk has called empathy “the fundamental weakness of Western civilization.” He sees it as a weapon wielded by liberal society to bludgeon otherwise rational people into operating against their own interests. Empathy is something done to you by others—a vulnerability they exploit, a back door through which they gain access to your resources and will. This rejection of empathy as a human value gives cover to people who don’t want to feel anything at all. If empathy is the problem, then lack of it isn’t a deficiency—it’s an advantage.

Elizabeth Bruenig: The conservative attack on empathy

I finally met Bezos on the last day of Campfire, at lunch, after my wife had broken her wrist. I went over to thank him for having us, and he asked how our Campfire experience had been. I told him that it was great, but that unfortunately my wife had broken her wrist that morning when she slipped on the wet grass while kicking a ball with our 6-year-old son.

The night before, we’d all stood by the pool at the beach club watching a cadre of synchronized swimmers execute a flawless water routine. I had spoken with a famous novelist, who said, “I just don’t understand why I’m here.” A famous rock star was about to start an acoustic set. The famous chef had made paella. Somewhere deep under my skin, a brutal pox was beginning to form.

The next morning, my wife fell, and I found myself in a black SUV with a team of private-security contractors, who whisked us to the back entrance of a Santa Barbara emergency room, where she was seen and treated right away. We made it back in time to watch the Supreme Court justice Zoom in from Washington, D.C.

How was your Campfire? Bezos asked me an hour later, and because I am an honest person, and because I have been a host myself, I decided he would want to know that there had been a problem, but that his team had reacted quickly and been extremely helpful. To be clear, I was in no way blaming him, nor was I shaking down the richest man on Earth. Instead, I was simply offering Bezos, also a husband and father, a brief human connection.

But when I told him what had happened, Bezos looked horrified. He did not say “I’m so sorry.” He did not say “Do you need anything?” Instead, he made a face, and in an instant, an aide came and whisked him away. When presented with the opportunity for empathy, even performative empathy, he chose escape.

A few hours later, on the private plane home, a famous movie producer offered my wife a blanket. My children’s faces were covered in spots. Under my fingernails, red welts were beginning to rise.

The world has always been run by rich men. The robber barons of the Gilded Age were known for their ruthlessness in the accumulation of wealth—hiring Pinkertons to shoot striking unionists. But they directly engaged with the world around them, using their wealth and power to muscle it into its most profitable form. And although today’s billionaires are clearly manipulating society to maximize their own profit, something else is also happening—a disassociation from the reality of cause and effect, from meaning and history. These men no longer feel the need to change the world in order to succeed, because their success is guaranteed, no matter what happens to the rest of us.

“I’m finished,” yells Daniel Plainview, perched happily on the polished floor of his own celestial kingdom. Though he has just committed a crime, he has never felt so free.

This article appears in the May 2026 print edition with the headline “Everything Is Free and Nothing Matters.”

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · [SOURCE](#)

An agent with grep does quite well at search.

It's shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you'll have built the best search engine.

There's Sutton's [bitter lesson](#) about unleashing raw compute on a problem. But there's a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google's algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might answer:

Question:

| What is a runner test?

Answer:

| Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that's literally an answer to the question.

But I wouldn't consider this "good documentation". It's not anchored to any concepts from the question. It's just an out of context bit of text.

If you wrote this for a coding agent, you'd probably organize the chunk to give hints about why it's useful.

| docs/tests/runner.md

| # Runner Tests

| ## Purpose

| We need to ensure the system runs end to end

| ## Usage

| Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.

| ## What is it

| A runner test actually interacts with real-life test bank accounts to ensure we can correctly deposit / withdraw money and that the full banking system works.

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

You learn that the trick to RAG isn't creating good question answering systems. Its writing better answers.

The enterprise search morass... is over?

Compare how coding agents program us to the eternal nightmare of your company's wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to 'SEO' the content for Confluence search.

If someone is 'lucky' enough to find your article, they can barely understand it. It's written in your team's jargon. And it's probably out of date anyways.

For these reasons, I always tell people they'd do better to write a public blog article. Blogs assume zero context. They're clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you're incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You're writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Enterprise search fails because there's no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn't just code generation. It's closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What's in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that's spammy and we should ban them.

That's too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what's acceptable. What's considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what's useful not malicious.

Give content creators feedback. Once you've identified what 'good' is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don't.

Steer content creators towards the content you want. Don't hope algorithms can overcome drivel and slop.

Your search engine isn't a bundle of algorithms ranking for users. It's a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at [Cheat at Search with Agents](http://maven.com/softwareDoug/cheat-at-search) to learn use agents in search. build better RAG and use LLMs in query understanding.

© 2026 SoftwareDoug LLC

Ownership

THORSTEN BALL · 08 JUL 2026 · [SOURCE](#)

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you...

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?
- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?
- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?
- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?
- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!
- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say “here's what I built, under these constraints, with these tradeoffs?”
- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?
- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?
- If you think your colleagues need to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.
- Do customers need to know? Who reported the bug? Who's blocked? Let them know.
- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



“How does this apply juniors? You can't expect them to really do all of that?”

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, ... — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

[Daily article] July 10: Gu Yanwu

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 10 JUL 2026 · [SOURCE](#)

Gu Yanwu (1613–1682) was a Chinese historian, philologist, and poet. Born in Qiandun, he pursued advancement in the imperial examination system but failed to achieve the rank of juren. He was a Ming loyalist after the Qing conquest and declined political positions to travel across China. His work, *Rizhilu*, is an edited collection of his notes on various topics, mainly statecraft and historiography. He was critical of Neo-Confucianism, Buddhism, Daoism, and the political centralization and reliance on law codes. He advocated for historical study centered on primary sources. *Rizhilu* and his phonology treatise *Yinxue wushu* were published during his lifetime; his other surviving works were published by Pan Lei after his death. Gu's thought influenced scholars throughout the Qing period, and 19th-century scholars such as He Shaoji venerated him at a Beijing temple constructed in his honor. Revolutionaries such as Liang Qichao praised his work, stressing his empiricism and resistance to Qing rule. (Full article...).

Read more: https://en.wikipedia.org/wiki/Gu_Yanwu

Today's selected anniversaries:

1553:

Lady Jane Grey was proclaimed the successor to King Edward VI of England, beginning her disputed reign as the "Nine Days' Queen". https://en.wikipedia.org/wiki/Lady_Jane_Grey

1921:

Irish War of Independence: One day after a truce was agreed between the Irish Republican Army and British forces, violence broke out between Catholics and Protestants in Belfast. https://en.wikipedia.org/wiki/Bloody_Sunday_%281921%29

1941:

The Holocaust: Ethnic Poles murdered at least 340 Jewish residents of Jedwabne in German-occupied Poland. https://en.wikipedia.org/wiki/Jedwabne_pogrom

2011:

The Russian river cruise liner Bulgaria was caught in a storm in Tatarstan on the Volga and sank in several minutes, resulting in 122 deaths. https://en.wikipedia.org/wiki/Bulgaria_%281955_ship%29

Wiktionary's word of the day:

misguided: 1. Of a person: lacking proper guidance; misled or mistaken in one's actions or thinking. 2. Of a plan, policy, etc.: not properly planned or thought through; ill-conceived. 3. (obsolete) 4. Of a government, etc.: mismanaged, misruled. 5. Of a person: behaving badly; immoral. 6. Poorly educated or trained. [...] 7. About Word of the Day 8. Nominate a word 9. Leave feedback <https://en.wiktionary.org/wiki/misguided>

Wikiquote quote of the day:

Once upon a time I was falling in love But now I'm only falling apart There's nothing I can do A total eclipse of the heart Once upon a time there was light in my life But now there's only love in the dark Nothing I can say A total eclipse of the heart. --Jim Steinman https://en.wikiquote.org/wiki/Jim_Steinman